Cinemath: Separating Pedagogical Reasoning from Deterministic Educational Animation

Mauricio Tedeschi

July 12, 2026

Abstract

Large language models can explain mathematical and physical reasoning, but they are unreliable authors of low-level animation code. We present Cinemath, an open-source pipeline that turns a typed or photographed problem into a short educational Manim video while keeping those roles strictly separated. An LLM acts only as a teacher: it produces a structured lesson plan consisting of conceptual steps and a list of named visual-tool calls. Local compilers then expand those calls into a validated animation script, optionally check arithmetic and symbolic claims with SymPy, and render the result with Manim Community. Because scene construction never depends on free-form model-generated Python, visual fidelity is controlled by hand-written templates spanning paper arithmetic, algebra boards, calculus plots, formal proofs, and quantum-field-theory diagrams including one-loop layouts and renormalization-group flow sketches. We describe the architecture, the plan schema, the visual-tool catalog, and a difficulty ladder from elementary arithmetic through Peskin-style β-function problems.

1 Introduction

Educational animation systems such as Manim make it possible to present equations, graphs, and diagrams with cinematic timing [1]. At the same time, frontier language models have become competent tutors for multi-step mathematics and physics [2]. Combining the two is attractive: given a homework-style prompt, one would like a narrated visual solution rather than a static wall of text.

A naïve approach—asking the model to emit Manim source directly—fails in predictable ways. Generated code frequently invents nonexistent APIs, breaks LaTeX mid-expression, hard-codes fragile layout constants, or silently skips pedagogically important beats. Even when the mathematics in the explanation is correct, the animation layer may not compile.

Cinemath adopts the opposite contract. The language model is restricted to *teaching*: it must return a versioned JSON plan with a normalized problem statement, a final answer, a short sequence of conceptual steps, and a non-empty list of visual tools drawn from a fixed catalog. All animation construction, validation, and rendering remain local and deterministic. This separation mirrors classical compiler design: the LLM emits an intermediate representation; trusted backends lower it to pixels.

The system is deliberately domain-agnostic at the plan level. Instead of a single mutually exclusive "problem type," the teacher selects a composition of tools (for example, a rectangular double integral may request a region outline, an equation board, a three-dimensional surface, and a final answer beat). Specialized QFT tools cover interaction Lagrangians, tree-level $1 \to 2$ decays, one-loop / 1PI diagrams, and two-dimensional renormalization-group flow fields of the kind appearing in textbook exercises [3].

2 Architecture

Figure 1 summarizes the end-to-end pipeline. A run begins with a text file, Markdown note, or image of a problem. Images are OCR'd / transcribed by the same model family used for teaching; the extracted statement is then passed to the teacher prompt. The teacher may call local arithmetic tools for long multiplication, division, addition, or subtraction so that digit-level results are ground-truth rather than model-invented. After the plan is accepted, a verifier inspects known visual-tool payloads (for example, quadratic coefficients and roots, or paper-arithmetic products). Local template compilers expand `visuals[]` into an internal animation DSL, prepend a problem-statement scene, and validate the script. Manim finally renders `animation.mp4` together with a Markdown lesson transcript.

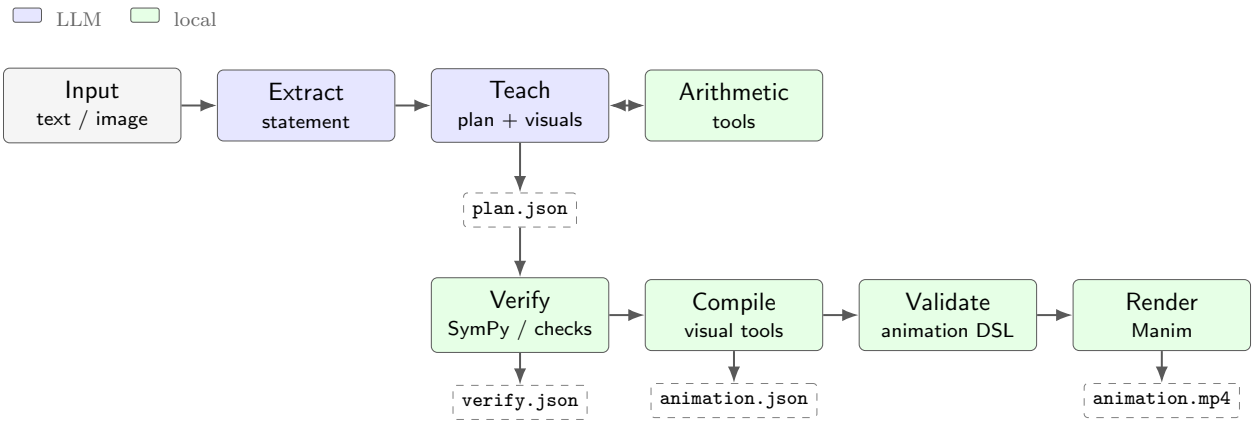


Figure 1: Cinemath pipeline. Blue stages may call the language model; green stages are fully local. Dashed boxes are on-disk artifacts written for every run. The teacher may call local arithmetic tools before emitting the final plan; it never authors Manim scenes.

2.1 Trust boundary

The critical invariant is that `animation.json` is never authored by the model. The teacher may choose *which* visual capabilities to invoke and fill their typed fields (coefficients, region bounds, Feynman labels, β-function expressions), but scene graph construction, caption timing, equation-chain morphs, and camera moves live in versioned Python modules under `templates/` and `render_engine/`.

2.2 Artifacts

Each successful `cinemath solve` run creates a timestamped directory containing

- `problem.txt` — normalized statement with ASCII LaTeX in `$...$`;
- `plan.json` — teacher plan (version 2);
- `verify.json` — local check summary;
- `lesson.md` — human-readable lesson transcript;
- `animation.json` — validated internal DSL;
- `scene.py` / `animation.mp4` — Manim entrypoint and rendered video.

2.3 Worked example: quadratic roots

Figure 2 and Listings 1–3 walk through a real run of `examples/04_quadratic.txt` ($x^2-5x+6=0$). The teacher returns the abbreviated plan in Listing 1 (full five-step explanations omitted with "..."). Local verification confirms the claimed roots against SymPy (Listing 2). The `plot_2d` compiler then expands the visual payload into the animation-DSL scene in Listing 3, which Manim renders as the parabola / root markers in Figure 3(b).

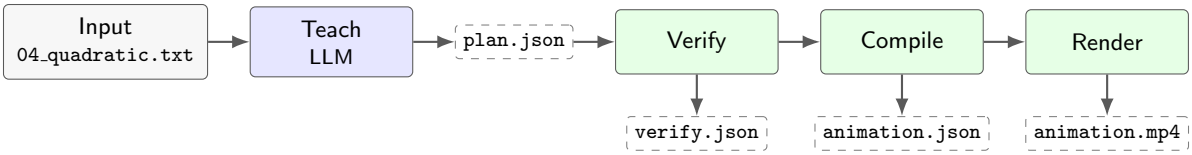


Figure 2: Artifact flow for the quadratic example. Blue is LLM-authored JSON; green stages are local. Excerpt files used below live under `docs/examples/quadratic/`.

Listing 1: Abbreviated teacher `plan.json` (visuals retained in full).

```
{
  "version": 2,
  "problem": "Solve for x: x^2 - 5x + 6 = 0",
  "answer": "x = 2, x = 3",
  "steps": [
    {
      "title": "Identify the quadratic",
      "explanation": "This is a quadratic equation in standard form $ax^{2}+bx+c=0$ with $a=1$,
    $b=-5$, $c=6$.",
      "math": ["x^2 - 5x + 6 = 0", "a=1,\\; b=-5,\\; c=6"]
    },
    {
      "title": "...",
      "explanation": "(three more factoring / verification steps omitted)",
      "math": ["..."]
    },
    {
      "title": "Verify the roots",
      "explanation": "Check by substitution that both roots satisfy the equation.",
      "math": ["2^2 - 5(2) + 6 = 0", "3^2 - 5(3) + 6 = 0"]
    }
  ],
  "visuals": [
    {
      "tool": "plot_2d",
      "equation": "x^2 - 5x + 6 = 0",
      "coefficients": {"a": 1.0, "b": -5.0, "c": 6.0},
      "roots": [2.0, 3.0]
    },
    {"tool": "equation_board"},
    {
      "tool": "show_answer",
      "tex": "x=2,\\; x=3",
      "caption": "Solutions to the quadratic"
    }
  ]
}
```

Listing 2: Local `verify.json`: SymPy agrees with the claimed roots.

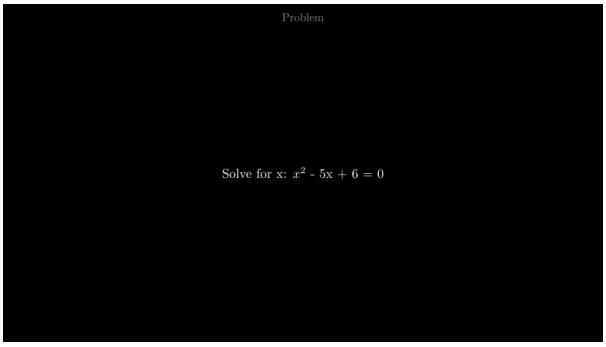

```
{
  "checked": true,
  "ok": true,
  "notes": [
    "Roots match SymPy."
  ],
  "tools": [
    "plot_2d",
    "equation_board",
    "show_answer"
  ],
  "computed_roots": [
    2.0,
    3.0
  ],
  "claimed_roots": [
    2.0,
    3.0
  ]
}
```


Listing 3: Compiled `plot_2d` scene from `animation.json` (axes, curve, and root dots).

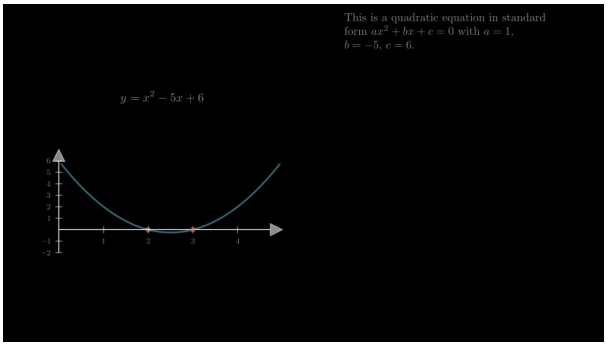

```
{
  "id": "plot_2d_0",
  "caption": "Where the parabola meets the x-axis",
  "objects": [
    {
      "id": "quad_lab_0",
      "type": "math",
      "color": "white",
      "font_size": 34.0,
      "at": "title",
      "tex": "y = x^2 - 5x + 6"
    },
    {
      "id": "quad_ax_0",
      "type": "axes",
      "color": "white",
      "at": [
        0.0,
        -0.9
      ],
      "x_range": [
        0.0,
        5.0,
        1.0
      ],
      "y_range": [
        -2.0,
        7.0,
        1.0
      ],
      "x_length": 6.5,
      "y_length": 3.0
    },
    {
```

```
      "id": "quad_curve_0",
      "type": "plot",
      "color": "blue",
      "axes": "quad_ax_0",
      "expr": "1.0*x**2 + (-5.0)*x + (6.0)",
      "x_range": [
        0.05,
        4.95
      ],
      "shade": "none"
    },
    {
      "id": "quad_root_0_0",
      "type": "dot",
      "color": "red",
      "at": [
        2.0,
        0.0
      ],
      "axes": "quad_ax_0"
    },
    {
      "id": "quad_root_0_1",
      "type": "dot",
      "color": "red",
      "at": [
        3.0,
        0.0
      ],
      "axes": "quad_ax_0"
    }
  ],
  "actions": [
    {
      "op": "write",
      "targets": [
        "quad_lab_0"
      ]
    },
    {
      "op": "create",
      "targets": [
        "quad_ax_0",
        "quad_curve_0"
      ]
    },
    {
      "op": "fade_in",
      "targets": [
        "quad_root_0_0",
        "quad_root_0_1"
      ]
    },
    {
      "op": "indicate",
      "targets": [
        "quad_root_0_0",
        "quad_root_0_1"
      ]
```

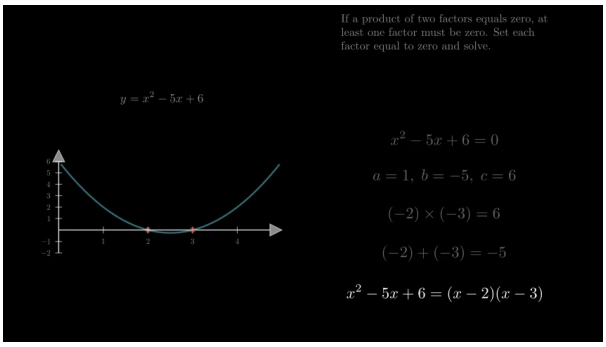
```
    },
    {
      "op": "wait",
      "seconds": 0.8
    }
  ]
}
```



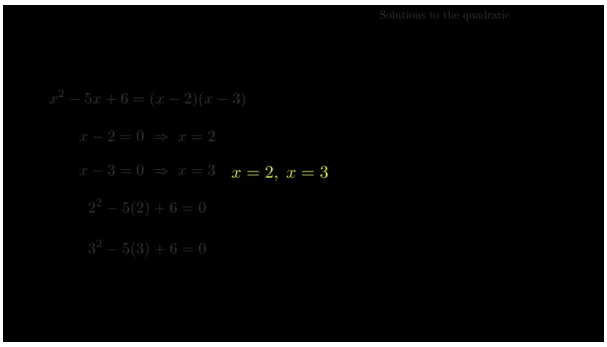
(a) Problem-statement beat from `problem.txt`.



(b) Frame from the compiled `plot_2d` scene.



(c) Equation board advancing through factoring steps.



(d) Final `show_answer` beat ($x = 2$, $x = 3$).

Figure 3: Rendered frames from the same quadratic run, in pipeline order. Source video: `outputs/.../animation.mp4`.

3 Teacher plan schema

Plans are JSON objects with schema version 2. The required fields are `problem`, `answer`, `steps`, and a non-empty `visuals` array. Each step carries a short board title, a one-to-three sentence explanation, and zero or more display-math fragments. Visual entries are tagged by `tool` and validated against a closed catalog (Section 4).

The system prompt forbids animation design language: the model must not mention Manim, colors, camera angles, or frame timing. Mathematical hygiene rules require ASCII LaTeX (no Unicode glyphs such as μ or ϕ written as code points) and consistent dollar wrapping in prose fields. Pure formula fields omit surrounding dollars so that Manim's `MathTex` path remains unambiguous.

For long arithmetic, the teacher must call the matching local tool (`long_multiply`, `long_divide`, `long_add`, or `long_subtract`) and copy the returned digits into both `answer` and the corresponding `paper_long_*` visual. This prevents the common failure mode in which a fluent explanation ends with an incorrect product.

4 Visual tools

Table 1 lists the current catalog. Tools are intentionally composable: a proof uses `state_claim`, `equation_board`, and `show_qed`; a rectangular double integral typically stacks region, board, surface, and answer beats; a Yukawa β-function exercise may combine a Lagrangian frame, one or more loop diagrams, the equation board, an RG flow field, and a highlighted answer.

Table 1: Visual-tool catalog exposed to the teacher (plan version 2).

Tool	Role
`equation_board`	Caption + stacked derivation lines
`state_claim` / `show_qed`	Proof opener / closer
`plot_2d`	Quadratic parabola with marked roots
`show_region_rectangle`	2D rectangular integration region
`plot_surface_3d`	Surface over a rectangular domain
`paper_long_*`	Stacked paper arithmetic layouts
`show_lagrangian`	Interaction / Lagrangian setup
`feynman_1to2`	Tree-level $1 \to 2$ decay diagram
`feynman_loop`	One-loop layouts (bubble, 4-point, Yukawa triangle)
`rg_flow_2d`	Arrow field from $\beta_x(x, y)$, $\beta_y(x, y)$
`show_answer`	Final highlighted answer beat

Each tool maps to a small compiler that emits objects in an internal DSL (`math`, `prose`, `axes`, `plot`, `feynman`, `flow_field`, ...) and a short action list (`write`, `create`, `indicate`, `wait`, ...). The renderer interprets that DSL only after schema validation, including safe-expression checks for plots and RG fields.

5 Rendering and pedagogy

Algebraic derivations use an equation chain: the previous line is dimmed, the board scrolls to make room, and a copy morphs into the next expression via Manim's `TransformMatchingTex`. Step explanations become wrapped instruction banners sized for full-frame or split-stage layouts. Every video opens with a problem-statement beat so the viewer sees the claim before the solution choreography begins.

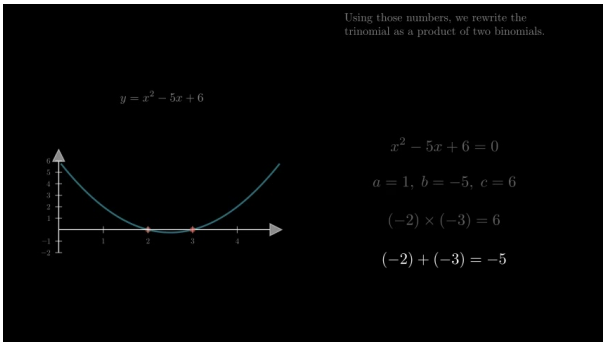
Two- and three-dimensional graph helpers auto-select axis ranges, optionally shade regions, and keep captions clear of plot chrome. QFT helpers draw schematic Feynman layouts and sample renormalization-group arrow fields from safe arithmetic expressions in variables (x, y), which the teacher uses as stand-ins for couplings such as (λ, g).

6 Examples

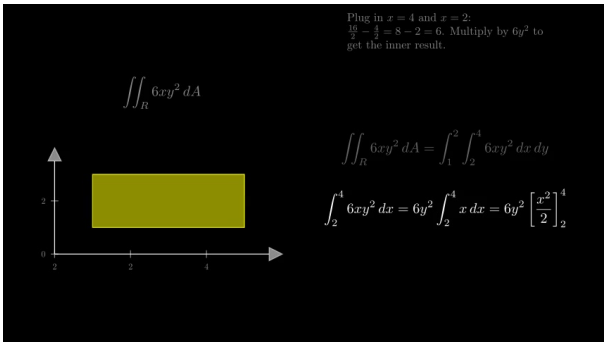
The repository ships a difficulty ladder from elementary word problems to tree-level scalar decay and related QFT exercises (Table 2). In practice the same architecture also covers photographed textbook problems, including Callan–Symanzik β-function computations in pseudoscalar Yukawa theory with qualitative RG-flow sketches [3]. Figure 4 shows representative frames from rendered lessons across several tool recipes.

Table 2: Representative example ladder shipped with the package.

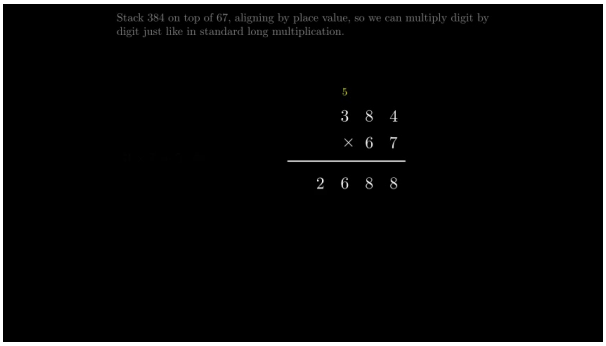
Level	Example	Typical visuals
1–3	Arithmetic / percent word problems	`equation_board`
4	Quadratic equation	`plot_2d`, board, answer
7	Rectangular double integral	region, board, surface, answer
8	Euler totient identity	claim, board, QED
10	Scalar $\Phi \to \phi\phi$ decay	Lagrangian, Feynman, board, answer
11–15	Paper long arithmetic	matching `paper_long_*`



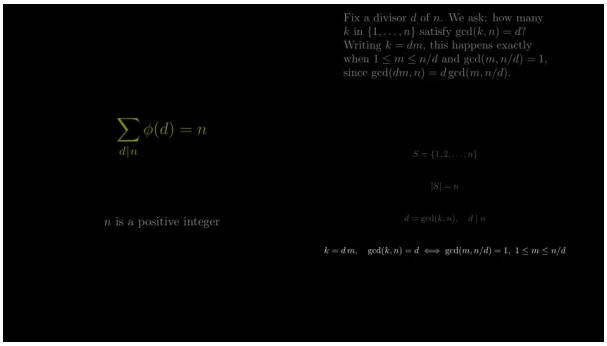
(a) Quadratic roots with `plot_2d` and equation board.



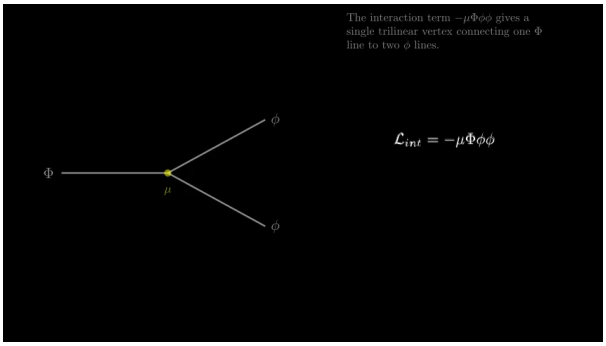
(b) Rectangular region with iterated double-integral board.



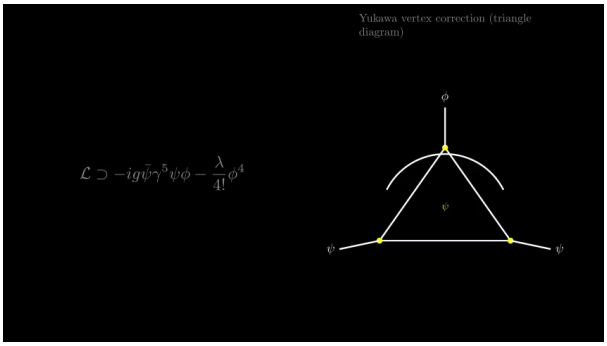
(c) Paper long multiplication with carry marks.



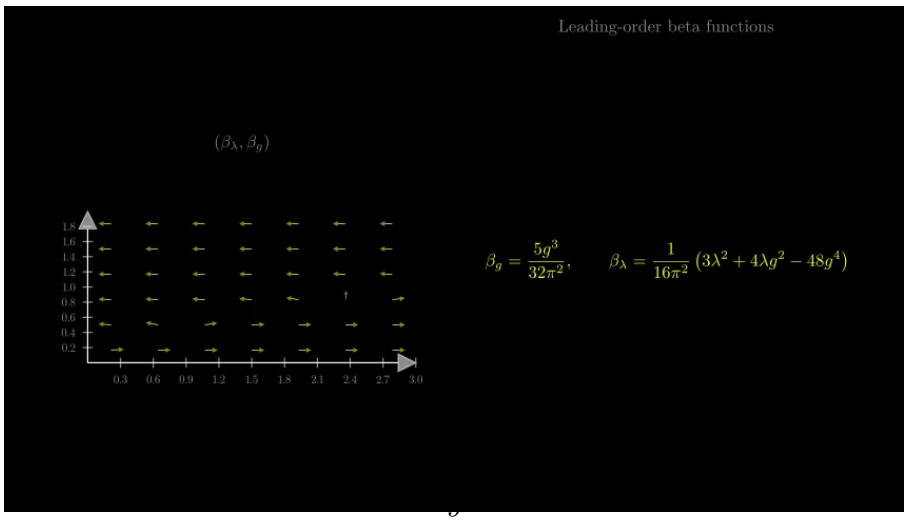
(d) Totient identity with claim opener and derivation board.



(e) Tree-level $1 \to 2$ decay diagram and interaction term.



(f) Yukawa Lagrangian with one-loop triangle layout.



(g) RG flow field (β_λ, β_g) with highlighted one-loop betas.

Figure 4: Frames from Manim lessons produced by CINEMATH. Each panel is a snapshot from a

7 Discussion

Restricting the model to a typed intermediate representation trades expressive freedom for reliability. New visual genres require new local compilers, but once written they are reused across every problem that selects them. The current catalog already tags tools by coarse domain (`core`, `proof`, `calculus`, `arithmetic`, `qft`) in preparation for a classify-then-gate prompt strategy that would expose only the relevant subset and thereby save context as the catalog grows.

Limitations remain. Verification coverage is uneven across tools; symbolic checks are strongest for arithmetic and selected algebraic payloads. Pedagogical quality still depends on the teacher's step decomposition. Schematic Feynman diagrams are illustrative rather than automated 1PI generators, and RG flow fields visualize provided β expressions rather than deriving them. Finally, image OCR inherits the usual transcription failure modes for dense blackboard photography.

Natural extensions include domain-gated tool catalogs, richer diagrammatic calculus, tighter SymPy verification of board lines, and optional narration audio aligned to scene captions.

8 Conclusions

Cinemath demonstrates a practical split between LLM pedagogy and deterministic educational animation. A language model teaches; local visual tools compile and Manim renders. The resulting pipeline produces short lesson videos across arithmetic, algebra, calculus, proof, and selected quantum field theory topics without asking the model to write animation code. Source code and examples are available in the accompanying repository.

Software

Implementation is in Python 3.12+ using the Anthropic API [2], SymPy [4], Typer, and Manim Community [1]. The command-line entrypoint is

```
uv run cinemath solve path/to/problem
```

Artifacts for each run are written under `outputs/`.

A Reproducibility

From the project root,

```
uv sync
cp .env.example .env   # set ANTHROPIC_API_KEY
uv run cinemath solve examples/04_quadratic.txt
```

References

[1] The Manim Community Developers. Manim – Mathematical Animation Framework (community edition). https://www.manim.community/, 2024. Software.

[2] Anthropic. Claude. https://www.anthropic.com/claude, 2024. Large language model.

[3] Michael E. Peskin and Daniel V. Schroeder. *An Introduction to Quantum Field Theory*. Addison-Wesley, Reading, MA, 1995.

[4] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017. doi: 10.7717/peerj-cs.103.